



Data Link Layer

Dr. Alekha Kumar Mishra



Data link layer

- Node-to-node communication using links
- Provide services to network layer
- The datagram from network layer is encapsulated at source host in a frame and decapsulated at the destination data link layer
- Important Services
 - Framing
 - Flow control
 - Error control
 - Congestion control
- Sub-layers of Data link layer
 - Data link control
 - Media Access Control (MAC)

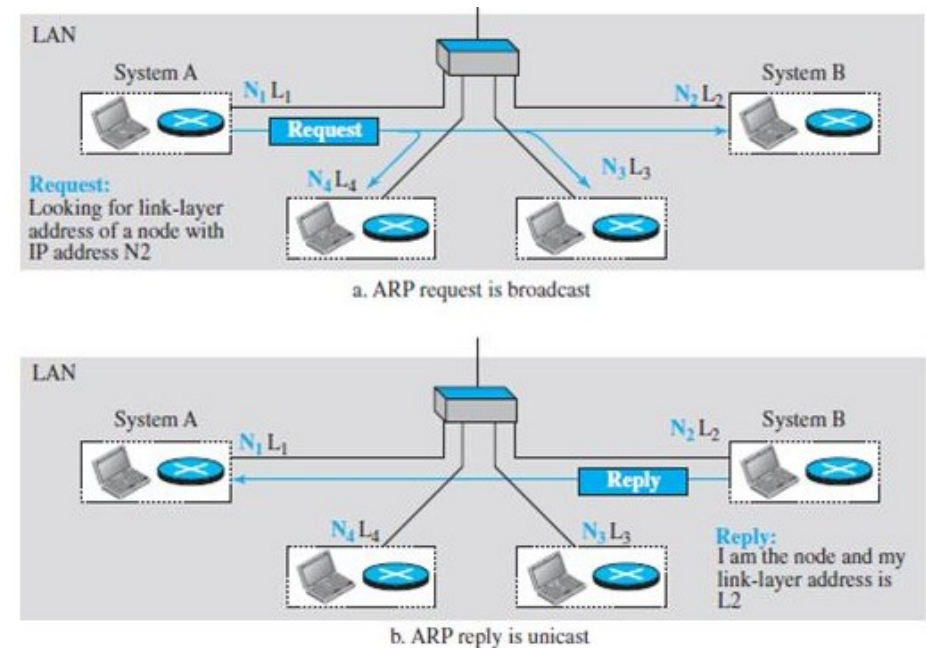


Link Layer Addressing

- IP addresses **cannot define links** through which datagram shall be sent as source and destination IPs **do not change** during transmission
- That's why link layer addressing (MAC address) is used for host-to-host communication
- Address types
 - Unicast address
 - Multicast address
 - Broadcast address

Address Resolution Protocol (ARP)

- A Protocol of network layer
- It maps an IP address to a logical-link address
- Steps
 - A host or router need to find link-layer address of another host broadcast an ARP Request in the network
 - The host that recognize the IP address sends back an ARP response packet.

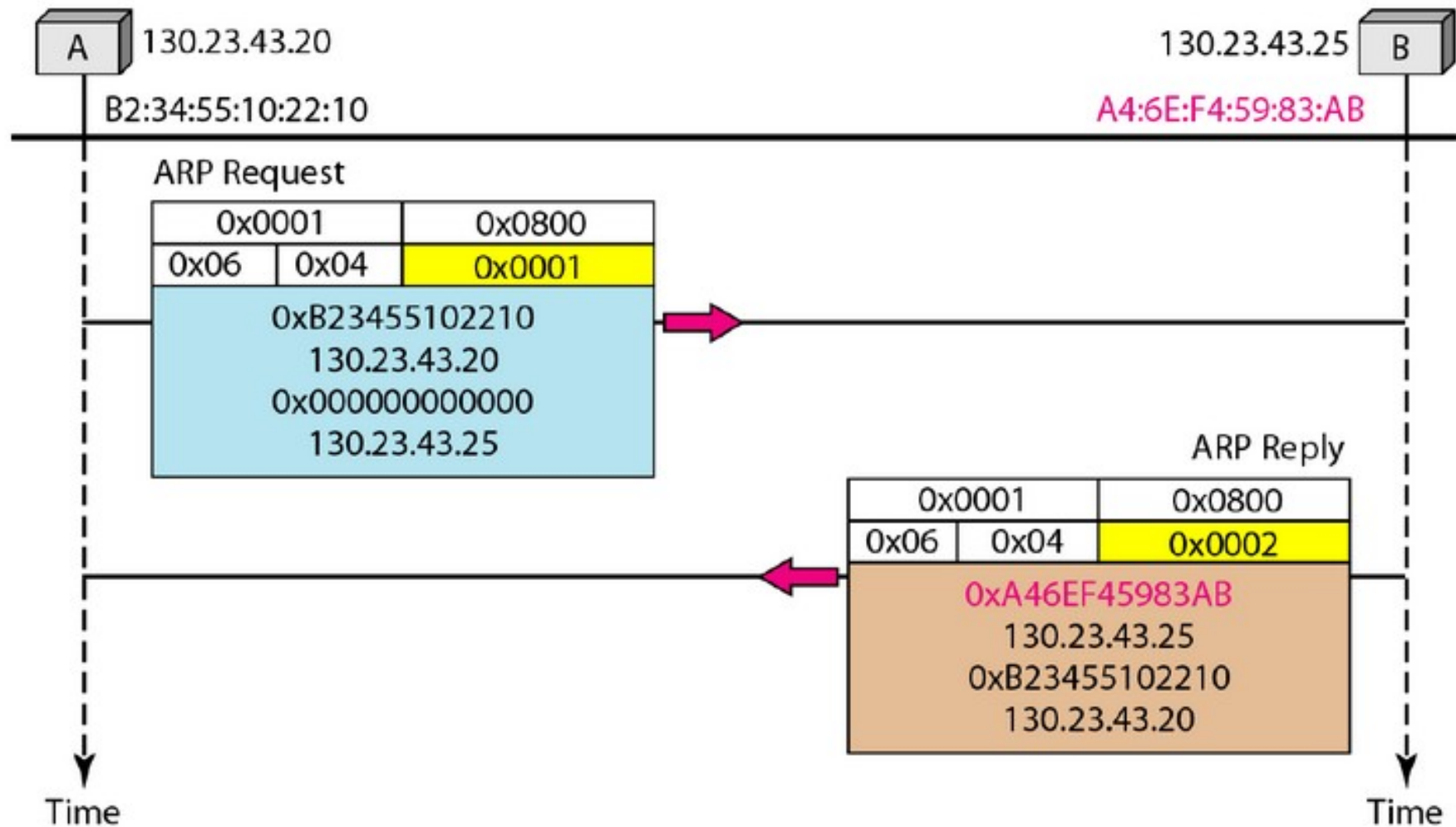




ARP Packet format

Hardware Type		Protocol Type
Hardware length	Protocol length	Operation Request 1, Reply 2
Sender hardware address (For example, 6 bytes for Ethernet)		
Sender protocol address (For example, 4 bytes for IP)		
Target hardware address (For example, 6 bytes for Ethernet) (It is not filled in a request)		
Target protocol address (For example, 4 bytes for IP)		

An example of ARP request and response





Error Detection and Correction



Introduction

- Data can be **corrupted** during transmission
- Some applications can **tolerate** a small level of error
- When we transfer text, we expect a very high level of **accuracy**
- Some applications require that errors be **detected** and **corrected**



Types of Errors

- **Single-Bit Error**

- The term single-bit error means that **only 1** bit of a given data unit is changed from 1 to 0 or from 0 to 1

- **Burst Error**

- The term burst error means that **2 or more** bits in the data unit have changed from 1 to 0 or from 0 to 1



Redundancy

- To be able to detect or correct errors, we need to send some **extra bits** with our data
- These redundant bits are added by the sender and removed by the receiver
- Their presence allows the receiver to detect or correct corrupted bits
- Redundancy is achieved through various coding schemes
 - **Block coding** and convolution coding



Detection Versus Correction

- The correction of errors is more **difficult** than the detection
- In error correction, we need
 - the **exact number** of bits that are corrupted
 - their **location** in the message
- To detect for k-bit error in n-bit data, there are nC_k possibilities

Forward Error Correction Versus Retransmission



- **Forward error correction** is the process in which the receiver tries to guess the message by using redundant bits
 - Suitable, if the number of errors is small
- **Correction by retransmission** is a technique in which the receiver detects the occurrence of an error and asks the sender to resend the message
 - Resending is repeated until a message arrives that the receiver believes is error-free

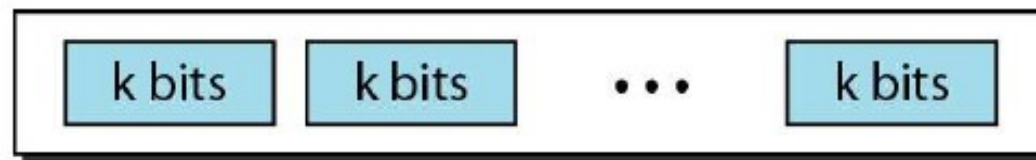


Mathematical Foundation

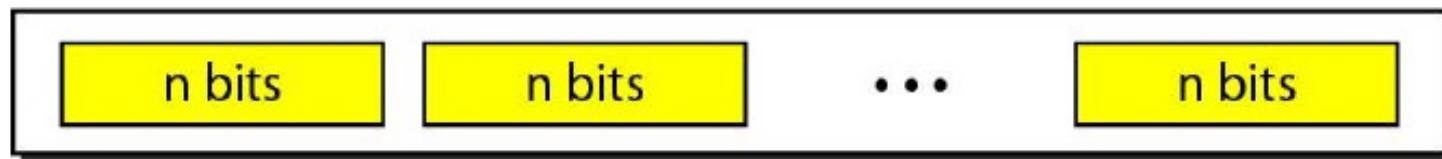
- Modulo 2 with operations + and *
- Denoted as $Z_2 = \{0, 1\}$

Block coding

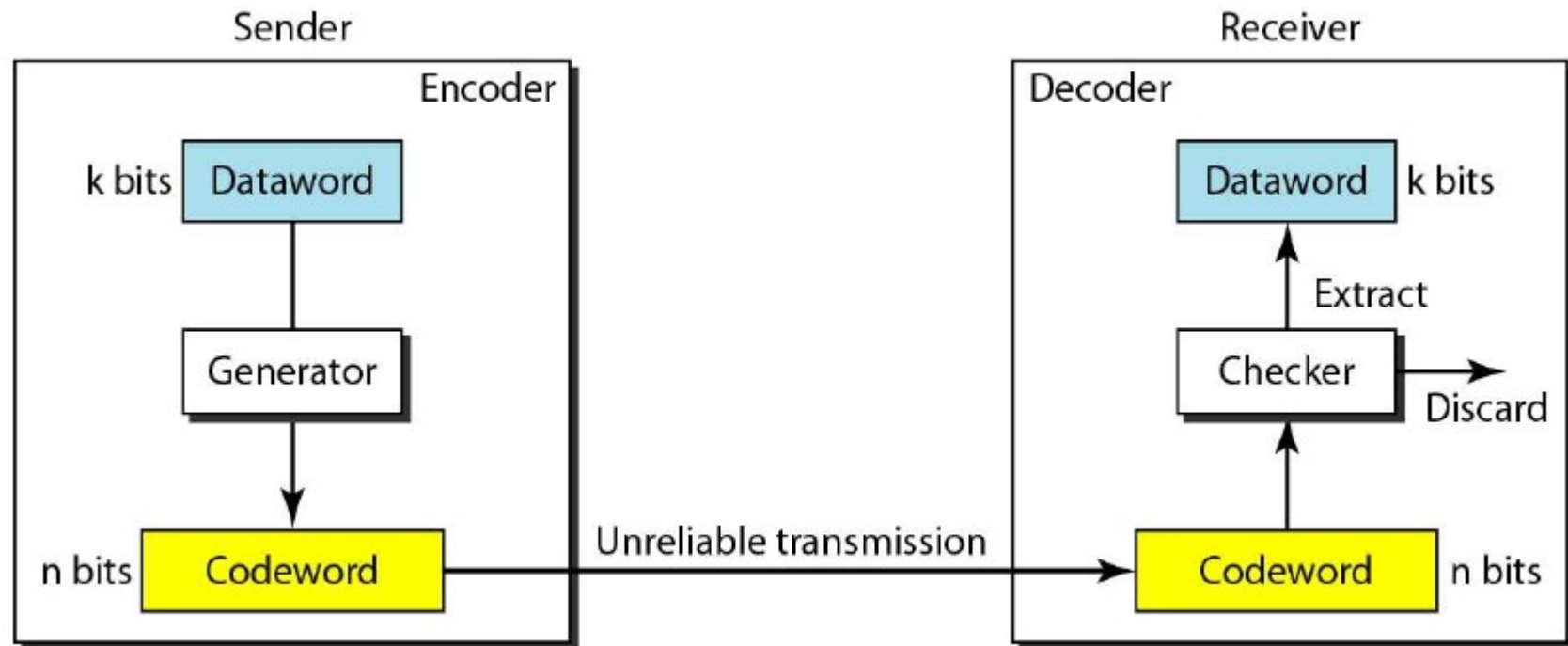
- We divide our message into blocks, each of k bits, called **datawords**
- We add r redundant bits to each block to make the length $n = k + r$
- The resulting n -bit blocks are called **codewords**.
- Unused $2^n - 2^k$ codewords called invalid or illegal



2^k Datawords, each of k bits



Block encoder and decoder





Error Detection : Hamming Distance

- The Hamming distance, $d(x,y)$ between two words x and y (of the same size) is the number of differences between the corresponding bits
- It determines the number of bits that are corrupted during transmission
 - (Hamming distance between sent codeword and received codeword)
- What is $d(00000,01101)$?
- What is $d(10101,11110)$?
- How can we implement this calculation in a programming code?



Minimum Hamming Distance

- In a set of words, the minimum Hamming distance $d_{\min}$ is the **smallest Hamming distance** between all possible pairs
- $d_{\min}$ is used to define the minimum Hamming distance in a coding scheme
- Find the minimum Hamming distance of the following set {10010, 01011, 11011, 11110}



Minimum Hamming distance

- A coding scheme C is written as $C(n, k)$ with a separate expression for $d_{\min}$
- n - size of codeword
- k – size of the dataword
- $d_{\min}$ – minimum hamming distance
- The theory
 - A Hamming distance s between the sent codeword and received codeword implies exactly s bit errors occurred
 - To detect up to s errors, the minimum distance between the valid codes must be $s + 1$
 - if the minimum distance between all valid codewords is $s + 1$, the received codeword cannot be erroneously mistaken for another codeword
 - Although a code with $d_{\min} = s + 1$ may be able to detect more than s errors in some special cases, only s or fewer errors are **guaranteed** to be detected.



Simple Parity-Check Code

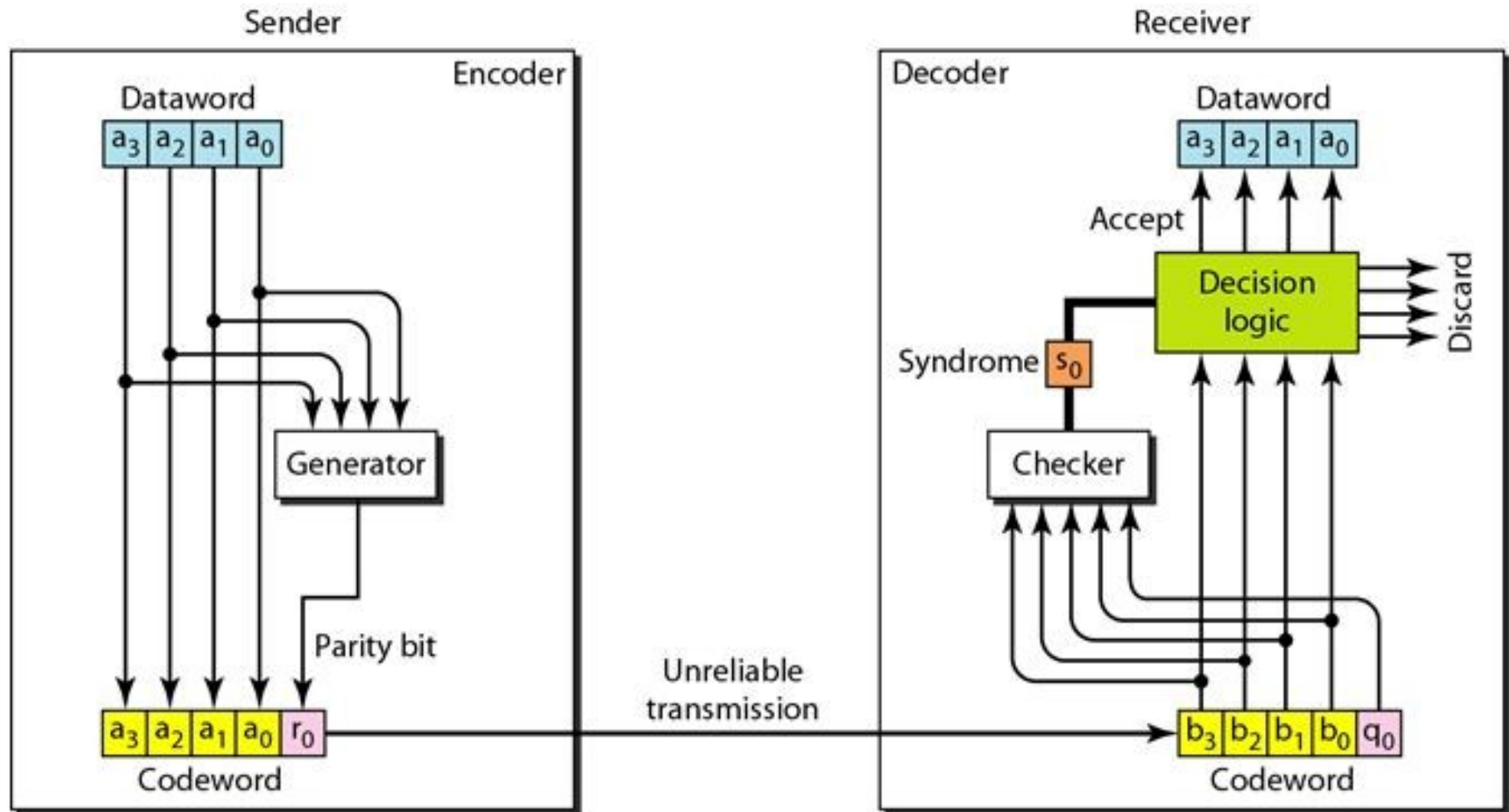
- $n = k + 1$
- The extra bit is called parity bit
- The bit is selected to make the total number of 1s in the codeword even
 - Some cases also specify an odd number of 1s, we discuss the even
- $d_{\min} = 2$



Example of Simple Parity-Check Code

<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
0000	0000 0	1000	1000 1
0001	0001 1	1001	1001 0
0010	0010 1	1010	1010 0
0011	0011 0	1011	1011 1
0100	0100 1	1100	1100 0
0101	0101 0	1101	1101 1
0110	0110 0	1110	1110 1
0111	0111 1	1111	1111 0

Simple Parity-Check encoder and decoder





Generating parity bit

- This is normally done by adding the 4 bits of the dataword (modulo-2)
- The **result** is the parity bit
- If the number of 1s is even, the result is 0
- If the number of 1s is odd, the result is 1.
- In both cases, the total number of 1s in the codeword is even
- The **syndrome** is 0 when the number of 1s in the received codeword is even; otherwise, it is 1.



Cyclic code

- In a **cyclic code**, if a codeword is cyclically shifted (rotated), the result is another codeword
- Example,
 - if 1011000 is a codeword, then 0110001 is also a codeword



Cyclic Redundancy Check (CRC)

- Also known as a polynomial code
- Based upon treating bit strings as representations of polynomials with coefficients of 0 and 1 only (Z_2).
- A k-bit frame is regarded as the coefficient list for a polynomial with k terms, ranging from x^{k-1} to x^0
- Such a polynomial is said to be of degree k – 1
 - For example, 110001 is represented as six-term polynomial $x^5 + x^4 + 1$.
- Polynomial arithmetic is done with modulo 2



Cyclic Redundancy Check (CRC)

- The sender and receiver must agree upon a generator polynomial, $G(x)$, in advance
 - Both the high- and low-order bits of the generator must be 1
- The length of textword polynomial $M(x)$ must be longer than the length of $G(x)$
- The idea is to append a CRC to the end of the frame in such a way that the polynomial represented by the checksummed frame is divisible by $G(x)$
- When the receiver gets the checksummed frame, it tries dividing it by $G(x)$. If there is a remainder, there has been a transmission error



Algorithm for CRC computation

- Let r be the degree of $G(x)$
- Append r zero bits to the low-order end of the frame so it now contains $m + r$ bits and corresponds to the polynomial $x^r M(x)$.
- Divide the bit string corresponding to $G(x)$ into the bit string corresponding to $x^r M(x)$, using modulo 2 division.
- Subtract the remainder from the bit string corresponding to $x^r M(x)$ using modulo 2 subtraction
- The result is the checksummed frame to be transmitted
- Call its polynomial $T(x)$



Algorithm for CRC computation

- Let r be the degree of $G(x)$
- Append r zero bits to the low-order end of the frame so it now contains $m + r$ bits and corresponds to the polynomial $x^r M(x)$.
- Divide the bit string corresponding to $G(x)$ into the bit string corresponding to $x^r M(x)$, using modulo 2 division.
- Subtract the remainder from the bit string corresponding to $x^r M(x)$ using modulo 2 subtraction
- The result is the checksummed frame to be transmitted
- Call its polynomial $T(x)$



Example

- Calculation for a frame 1101011111 using the generator $G(x) = x^4 + x^3 + x^2 + 1$

Frame: 1 1 0 1 0 1 1 1 1 1
Generator: 1 0 0 1 1

1 0 0 1 1 $\overline{)$ 1 1 0 1 0 1 1 1 1 1 0 0 0 0 $\leftarrow$ Quotient (thrown away)
1 0 0 1 1 $\leftarrow$ Frame with four zeros appended

1 0 0 1 1
1 0 0 1 1
0 0 0 0 1
0 0 0 0 0
0 0 0 1 1
0 0 0 0 0
0 0 1 1 1
0 0 0 0 0
0 1 1 1 1
0 0 0 0 0
1 1 1 1 0
1 0 0 1 1
1 1 0 1 0
1 0 0 1 1
1 0 0 1 0
1 0 0 1 1
0 0 0 1 0
0 0 0 0 0
1 0 $\leftarrow$ Remainder

Transmitted frame: 1 1 0 1 0 1 1 1 1 1 0 0 1 0 $\leftarrow$ Frame with four zeros appended minus remainder



Class exercise

- Given the dataword 1010011110 and the divisor 10111,
 - Show the generation of the codeword at the sender site (using binary division).
 - Show the checking of the codeword at the receiver site (assume no error)



Analysis

- Let $s(x) = c_r(x)/g(x)$
- If $s(x) \neq 0$, one or more bits is corrupted.
- If $s(x) = 0$, either
 - No bit is corrupted. or
 - Some bits are corrupted, but the decoder failed to detect them.
- Let Received codeword = $c(x) + e(x)$
- Received codeword/ $g(x) = c(x)/g(x) + e(x)/g(x)$
- Either $e(x)$ is 0 or $e(x)$ is divisible by $g(x)$
- The later case is very important
- Those errors that are divisible by $g(x)$ are not caught.



Checksum

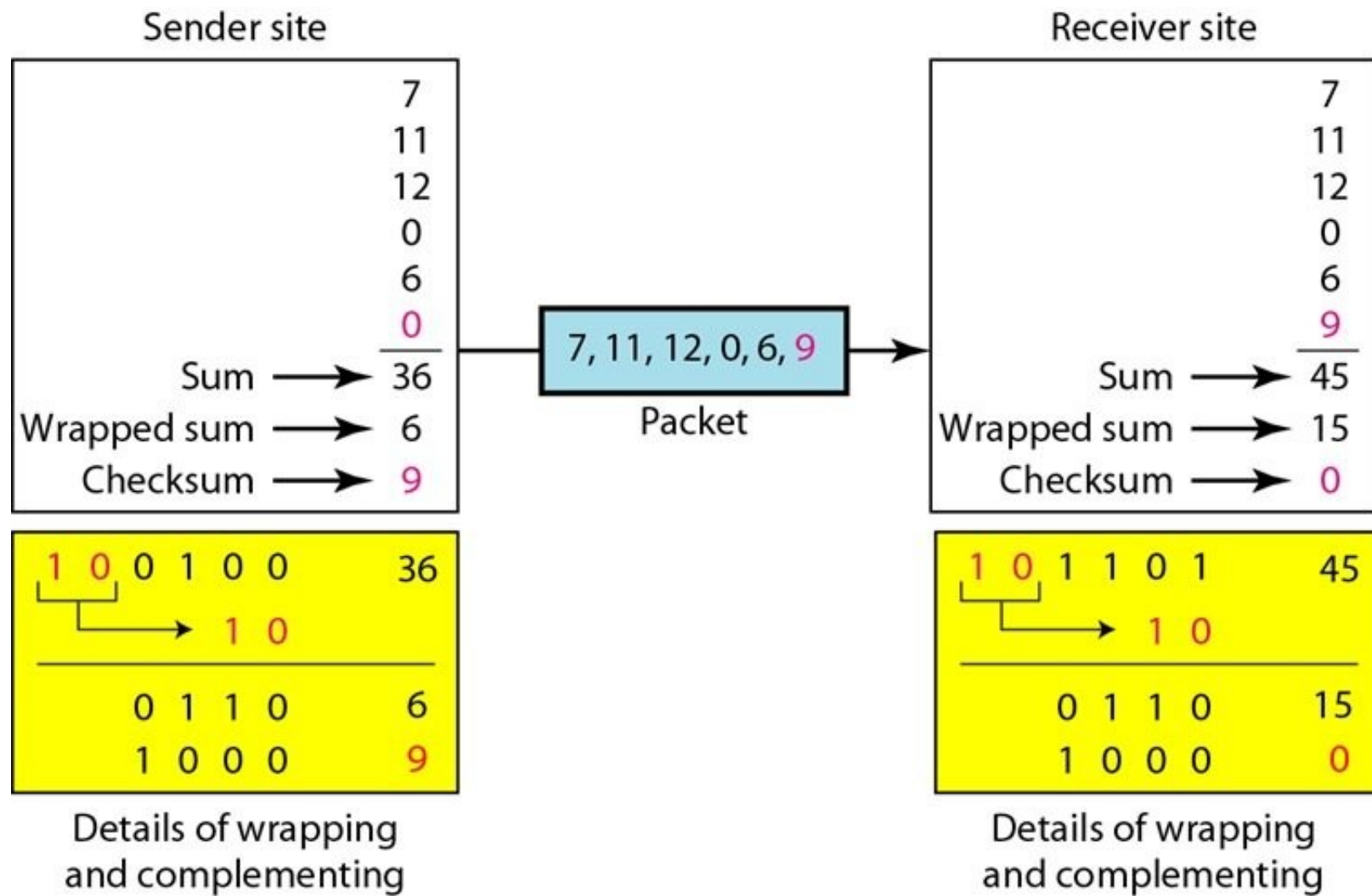
- Checksum is used by several layers
- Source message is divided into **m-bit** units
- An additional m-bit called **checksum** is created by generator and sent with the message
- The checker at destination create a new checksum using message and sent checksum
- If new checksum **zero**, then accept else an error is detected



One's complement addition

- $A = 10, B = 7$
 - $A = 1010, B = 0111$
 - $A + B = 1010$
 - $\quad + \quad 0111$
 - -----
 - $\quad 10001$
- $A + B = 0001 + 1$
 $= 0010$ (Ans)

Checksum example





Internet Checksum Algorithm

- Sender site:
 - The message is divided into **16-bit** words.
 - The value of the checksum word is set to 0.
 - All words including the checksum are added using **one's complement addition**.
 - The sum is **complemented** and becomes the **checksum**.
 - The checksum is sent with the data.
- Receiver site:
 - The message (including checksum) is divided into **16-bit** words.
 - All words are **added** using **one's complement addition**.
 - The sum is **complemented** and becomes the new checksum.
 - If the value of checksum is **0**, the message is **accepted**; otherwise, it is **rejected**.



Two-dimensional parity

- For single bit detection as well as correction
- Textword=1011 0100 1101 1001
- Let the width = 4
- Arrange the textword in matrix format with 4 columns
- 1011
- 0100
- 1101
- 1001
- Calculate parity for each row and column
- 1011**1**
- 0100**1**
- 1101**1**
- 1001**0**
- **10111**
- Send the codeword
1011**1**0100**1**1101**1**1001**010111**
- Let received codeword is
- 1011**1**0110**1**1101**1**1001**010111**
- Check parity for each row and column
- 1 0 1 1 **1** Y
- 0 1 **1** 0 **1** N
- 1 1 0 1 **1** Y
- 1 0 0 1 **0** Y
- **1 0 1 1 1** Y
- Y Y N Y Y
- The 2nd row and 3rd column does not have parity
- Hence, bit at (2,3) is corrupted



Error Correction using Hamming codes

- To guarantee the correction of t -errors in all cases, the minimum hamming distance in a codeblock must be $d_{\min} = 2t + 1$
- For error correction, no. of additional bits should be added such that it can point the position of the bit error
- If k is the number of additional bits to be used for error correction of m data bits, then the following must hold true
 - $(m + k + 1) \leq 2^k$
- Given m , this puts a lower limit on the number of check bits needed to correct single errors



Error Correction using Hamming codes

- To each group of m -data bit, k parity bits are added to form $(m+k)$ bit codeword.
- K -parity bits are placed in position 1,2,4,8,16,32,...(multiple of 2)
- Let $c_1, c_2, \dots, c_m$ indicate the text bits and let $p_1, p_2, p_4, \dots, p_i$ indicate the parity bits in the codeword
- Paritys are generated in the following way
- p_1 – start from position 1 in C, take 1 bit and skip 1 bit till all bits are processed
- p_2 - start fro position 2 in C, take 2 bits and skip 2 bits till all bits are processed
- p_4 - start from poosition 4 in C,take 4 bits and skip 4 bits till all bits are processed
- So on .. for all parity bit
- Select and put parity bit in C to generate final C



Example

- $D=01001101$
- $k=4$
- Then, $C=p_1p_20p_4100p_81101$
- $p_1-(p_10010)=1$
- $p_2-(p_21010)=0$
- $p_4-(p_40110)=0$
- $p_8-(p_81)=1$
- $C=\mathbf{100010011101}$



Example

- $C = 10001\mathbf{1}011101$
- $p1 - (p_1 0\mathbf{1}10) = 1 \text{ N}$
- $p2 - (p_2 1\mathbf{1}10) = 0 \text{ N}$
- $p4 - (p_4 0110) = 0 \text{ Y}$
- $p8 - (p_8 1) = 1 \text{ Y}$
- The position of bit corrupted is $1 + 2 = 3$



Next : data link control